02-16、自动化测试Agent开发之智能体交互

1、智能体问题优化

1、意图识别重试机制

```
def intent_recognition(self):
    """意图识别函数"""
    max_retries = 3
    for attempt in range(max_retries):
        try:
            tools_info = self.get_tools_info()
            chain = intent_recognition_prompt | deepseek_v3_llm |
self.intent_parser
            intent_result = chain.invoke({"user_input": self.user_input,
"tool_info": tools_info})
            self.intent_result = intent_result
            print("意图识别结果:", intent_result)
            return
        except Exception as e:
            print(f"意图识别失败 (第{attempt+1}次尝试): {e}")
            time.sleep(2 ** attempt)
```

2、完善工具参数的校验

问题:

- 工具参数验证缺失
- 恶意输入可能导致系统异常

```
    # 验证参数格式
    try:
        tool = self.tools[tool_name]
        if tool.args_schema:
            tool.args_schema(**tool_args)
        return True
    except Exception as e:
        print(f"工具参数验证失败: {e}")
        return False
```

2、智能体交互页面设计

3、Agent交互API设计

实现流式执行Agent任务，前端实时显示进度数据

1、请求信息

> **接口信息**
>
> - 请求方式: POST
> - 请求路径: /agent/execute/stream
> - 功能描述: 实时流式执行Agent任务，通过Server-Sent Events (SSE)返回执行过程
>
> **请求类型**
>
> Content-Type: application/json
>
> **请求参数**

```
{
  "user_input": "查询web测试平台用户注册的接口，生成测试用例",
  "user_id": "demo_user",
}
```

参数名	类型	必填	长度限制	描述
user_input	string	是	1-1000字符	用户输入的任务描述
user_id	string	否	-	用户ID，用于会话管理，默认为"anonymous"

2、响应格式

> **响应格式**
>
> Content-Type: text/event-stream
>
> 响应采用SSE格式，每条消息格式为：
>
> ```
> data: {"type": "事件类型", "message": "消息内容", ...}
> ```

3、流式响应事件类型 (SSE Events)

事件类型	描述	触发时机	包含数据字段
start	任务开始事件	Agent任务开始执行时	type, message, session_id, timestamp
progress	进度更新事件	任务执行过程中的各个阶段	type, step, message, progress
intent_result	意图识别结果	意图识别完成后	type, data, progress
execution_plan	执行计划事件	任务规划完成后	type, data, progress
step_start	步骤开始事件	每个执行步骤开始时	type, step_info, step_number, total_steps, progress
step_complete	步骤完成事件	每个执行步骤完成时	type, step_info, step_number, result, progress
complete	任务完成事件	整个任务执行完成时	type, message, data, progress
error	错误事件	任务执行出现错误时	type, message, error, confidence(可选)

4、使用fastapi开发sse服务

- **安装依赖**

```
# 安装FastAPI和相关依赖
pip install fastapi uvicorn
```

1、 什么是Server-Sent Events (SSE)?

定义： SSE是一种Web标准，允许服务器向客户端推送实时数据流。

特点：

- 单向通信（服务器 → 客户端）
- 基于HTTP协议
- 自动重连机制
- 轻量级，易于实现

2 、SSE vs WebSocket vs 轮询

特性	SSE	WebSocket	轮询
通信方向	单向	双向	双向
协议	HTTP	WebSocket	HTTP
复杂度	简单	中等	简单
重连	自动	手动	不需要
服务器资源	低	中	高

3、 SSE数据格式

```
data: 这是消息内容
event: custom-event
id: 123
retry: 3000

data: {"type": "message", "content": "Hello World"}
```

4、最简单的SSE服务

```
# main.py
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import asyncio
import json
import time

app = FastAPI(title="SSE入门示例")


@app.get("/agent/execute")
async def stream_events():
    """基础SSE端点"""
```

```
    async def event_generator():
        counter = 0
        while True:
            # SSE数据格式
            data = {
                "session_id": '1234567',
                "message_id": counter
                "type": "test",
                "content": f"消息 #{message_count}",
                "timestamp": time.time()
            }
            # 构造SSE格式
            yield f"data: {json.dumps(data, ensure_ascii=False)}\n\n"
            counter += 1
            await asyncio.sleep(1)  # 每秒发送一条消息

    return StreamingResponse(
        event_generator(),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "Connection": "keep-alive",
            "Access-Control-Allow-Origin": "*",
        }
    )

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

5、参数的定义

```
from fastapi import FastAPI, Query
from pydantic import BaseModel
from typing import Optional
import uuid

app = FastAPI()

class StreamConfig(BaseModel):
    interval: float = 1.0
    max_messages: Optional[int] = None
    message_type: str = "info"

@app.post("/stream")
async def custom_stream(config: StreamConfig):
    """可配置的SSE流"""

    async def event_generator():
        session_id = str(uuid.uuid4())
        message_count = 0

        # 发送开始事件
        yield f"event: start\n"
        yield f"data: {json.dumps({'session_id': session_id, 'config':
config.dict()})}\n\n"
```

```
        while True:
            if config.max_messages and message_count >= config.max_messages:
                # 发送结束事件
                yield f"event: end\n"
                yield f"data: {json.dumps({'session_id': session_id,
'total_messages': message_count})}\n\n"
                break

            # 发送数据事件
            data = {
                "session_id": session_id,
                "message_id": message_count,
                "type": config.message_type,
                "content": f"消息 #{message_count}",
                "timestamp": time.time()
            }

            yield f"event: message\n"
            yield f"id: {message_count}\n"
            yield f"data: {json.dumps(data, ensure_ascii=False)}\n\n"

            message_count += 1
            await asyncio.sleep(config.interval)

    return StreamingResponse(
        event_generator(),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "Connection": "keep-alive",
            "Access-Control-Allow-Origin": "*",
        }
    )
```